UNIVERSIDAD NACIONAL DE LOJA · CARRERA DE COMPUTACIÓN · 2.º CICLO

Cuaderno de síntesis

Unidad 1 · Semanas 1 a 4

Lógica combinacional con diseño clásico

Nombre completo: Matías Santiago Calva González

Ciclo: 2.º ciclo · Carrera de Computación

Asignatura: Diseño de Circuitos

Docente: Ing. Luis Darío Sinche Cueva, Mg.

Trabajo individual · Resultado de aprendizaje R1: diseña circuitos combinacionales básicos con diseño clásico.

Mapa de la unidad

La Unidad 1 recorre el camino completo del **diseño clásico**: del bit a la compuerta, de la compuerta a la expresión, de la expresión a la expresión *mínima* y de ahí a circuitos reales como el sumador.

Sem.	Tema	Idea que me llevo
1	Sistemas numéricos, códigos y compuertas	Todo es 0 y 1; restar es sumar el negativo (complemento a dos); 7 compuertas y su tabla.
2	Álgebra de Boole y formas canónicas	Circuito = expresión = tabla de verdad. Σm (filas con 1) y ΠM (filas con 0). Simplificar ahorra chips.
3	Mapas de Karnaugh, indiferentes, NAND/NOR	Agrupar 1 en potencias de 2 da el mínimo; las X ayudan; SOP→NAND-NAND, POS→NOR-NOR.
4	Circuitos aritméticos	Semisumador, sumador completo, ripple, sumador/restador con XOR, overflow y comparador → ALU.

Flujo de diseño clásico (hilo conductor): enunciado → tabla de verdad → Σm / ΠM → simplificación (álgebra o Karnaugh) → expresión mínima → circuito (AND-OR, NAND-NAND o NOR-NOR) → costo en compuertas y circuitos integrados (CI).

Semana 1 · Sistemas numéricos, códigos y compuertas

1.1 Por qué binario

- Un transistor es un interruptor controlado por voltaje: dos estados (0 y 1) son fáciles de distinguir aunque haya ruido.
- Un «1» es un **rango** de voltaje. En 74LS: salida 1 ≥ 2,7 V, salida 0 ≤ 0,5 V; la entrada lee 1 si ≥ 2,0 V y 0 si ≤ 0,8 V. Entre 0,8 y 2,0 V es **zona indefinida** (típico de una entrada sin conectar).
- **Margen de ruido**: alto = 2,7 − 2,0 = 0,7 V; bajo = 0,8 − 0,5 = 0,3 V.

1.2 Sistemas posicionales y conversiones

- Cada posición vale una potencia de la base. Pesos binarios: 128 · 64 · 32 · 16 · 8 · 4 · 2 · 1. Bit = 1 dígito; byte = 8 bits.
- **Decimal → binario**: dividir entre 2 y leer los restos de abajo hacia arriba. Ej.: 25 = 11001_2 (16 + 8 + 1).
- **Hexadecimal**: grupos de 4 bits desde la derecha (A=10 … F=15). **Octal**: grupos de 3 bits.
- Ej.: 157 = 10011101_2 = $9D_{16}$ = 235_8.

1.3 Suma y números con signo

- Suma binaria: 0+0=0 · 0+1=1 · 1+1=10 (escribo 0, llevo 1) · 1+1+1=11. Ej.: 1011 + 0110 = 10001 (11 + 6 = 17).
- **Complemento a dos**: el bit izquierdo indica el signo. Negativo = **invertir los bits y sumar 1**. Con 4 bits el rango es −8 a +7 (8 bits: −128 a +127).
- Ej.: 5 − 3 = 5 + (−3) → 0101 + 1101 = 1 0010 → se descarta el 1 sobrante → 0010 = 2 ✓.
- **Desbordamiento**: 0111 + 0001 = 1000 = −8 ✗ (el resultado no cabe). Igual que un *int* de 32 bits en Java.

1.4 Códigos

Código	Qué es	Para qué sirve
BCD	Cada dígito decimal en 4 bits (157 → 0001 0101 0111). Usa 0000–1001; 1010–1111 no se usan.	Displays, relojes, contadores
Gray	Entre vecinos cambia **un solo bit** (00-01-11-10). Se construye con el método del espejo.	Encoders, perillas; orden del mapa de Karnaugh
ASCII	Tabla de 7 bits (128 caracteres): «A» = 65 = 0x41 = 01000001. Sin ñ ni tildes → UTF-8.	Texto

1.5 Las 7 compuertas

A	B	NOT A	AND	OR	NAND	NOR	XOR	XNOR
0	0	1	0	0	1	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	1	0	0	0	1

- AND: 1 solo si **todas** valen 1 · OR: 1 si **al menos una** vale 1 · XOR: 1 si las entradas son **distintas** (luz de escalera) · XNOR: 1 si son **iguales**.
- El círculo en la salida significa «negado». Con solo NAND (o solo NOR) se construye cualquier circuito.
- **Circuito integrado 74LS08** (4 AND de 2 entradas, DIP-14): VCC = pin 14 (+5 V), GND = pin 7; pin 1 junto a la muesca, conteo antihorario. Código SN 74 LS 08 N: fabricante · rango comercial · familia (LS = TTL, HC = CMOS) · función · encapsulado.

Semana 2 · Álgebra de Boole y formas canónicas

2.1 Circuito = expresión

Cable de entrada = variable · AND = producto (·) · OR = suma (+) · NOT = complemento (') · Precedencia: **NOT → AND → OR**. Ej.: Y = A·B + C'. Dos expresiones son equivalentes si y solo si tienen la misma tabla de verdad.

2.2 Postulados y dualidad

Postulado	Forma OR	Forma AND (dual)
Identidad	X + 0 = X	X · 1 = X
Complemento	X + X' = 1	X · X' = 0
Conmutativa	X + Y = Y + X	XY = YX
Distributiva	X(Y+Z) = XY + XZ	**X + YZ = (X+Y)(X+Z)**

Dualidad: intercambiar + ↔ · y 0 ↔ 1 (sin complementar variables) mantiene válida la identidad.

2.3 Teoremas de consulta

Teorema	Forma OR	Forma AND
Elemento nulo	X + 1 = 1	X · 0 = 0
Idempotencia	X + X = X	X · X = X
Involución	(X')' = X	
Absorción	X + XY = X	X(X + Y) = X
Abs. del complemento	X + X'Y = X + Y	X(X' + Y) = XY
Adyacencia	XY + XY' = X	(X+Y)(X+Y') = X
Consenso	XY + X'Z + YZ = XY + X'Z	(X+Y)(X'+Z)(Y+Z) = (X+Y)(X'+Z)

2.4 De Morgan: «rompe la barra, cambia el operador»

(AB)' = A' + B' · (A + B)' = A'B' · generaliza: (A + B + C)' = A'B'C'. Ej.: (A' + BC)' = A·(BC)' = A(B' + C') = AB' + AC'.

2.5 Minitérminos y maxitérminos (3 variables)

i	ABC	m_i (vale 1)	M_i (vale 0)
0	000	A'B'C'	A + B + C
1	001	A'B'C	A + B + C'
2	010	A'BC'	A + B' + C
3	011	A'BC	A + B' + C'
4	100	AB'C'	A' + B + C
5	101	AB'C	A' + B + C'
6	110	ABC'	A' + B' + C
7	111	ABC	A' + B' + C'

m_i: producto; variable en 1 → sin complemento, en 0 → complementada.

M_i: suma; variable en 0 → sin complemento, en 1 → complementada.

Relación: $M_i = (m_i)'$.

SOP = Σm: se leen las filas con F = 1 y se unen con OR.

POS = ΠM: se leen las filas con F = 0 y se unen con AND.

Los índices de ΠM son los que faltan en Σm.

2.6 Ejemplo completo: alarma de 3 sensores (suena si al menos dos se activan)

Cuidado con el enunciado: «exactamente dos» = Σm(3,5,6); «al menos dos» = Σm(3,5,6,7) (la que pide el cliente).

F = Σm(3,5,6,7) = A'BC + AB'C + ABC' + ABC = ΠM(0,1,2,4).

Simplificando con idempotencia (ABC se usa tres veces): A'BC + ABC = BC · AB'C + ABC = AC · ABC' + ABC = AB → **F = AB + AC + BC** (POS: (A+B)(A+C)(B+C)).

Costo: canónica = 7404 + 2×7411 + 7432 = **4 CI** · simplificada = 7408 + 7432 = **2 CI**.

2.7 Simplificación algebraica y criterio SOP/POS

- Procedimiento: pares adyacentes (XY + XY' = X) → duplicar términos si hace falta (X = X + X) → absorción → absorción del complemento → consenso. No hay algoritmo que garantice el mínimo: por eso existen los mapas de Karnaugh.
- Ej.: F = A + A'B + AB'C = (A + B) + AB'C = A + B (de 5 literales a una OR de 2 entradas).
- **¿SOP o POS?** Pocos 1 → SOP · pocos 0 → POS · solo NAND → SOP (NAND-NAND, 7400) · solo NOR → POS (NOR-NOR, 7402). El criterio final es el costo.

Semana 3 · Mapas de Karnaugh, indiferentes y NAND/NOR

3.1 Idea clave y estructura del mapa

Si dos minitérminos difieren en **una sola variable**, esa variable desaparece (XY + XY' = X). El mapa ordena las celdas en **código Gray** (00-01-11-10) para que los vecinos físicos sean vecinos lógicos, y los bordes opuestos también lo son (el mapa se enrolla como un toroide).

A\BC	00	01	11	10
0	0	1	3	2
	0	0	0	0
1	4	5	7	6
	0	0	0	0

AB\CD	00	01	11	10
00	0	1	3	2
	0	0	0	0
01	4	5	7	6
	0	0	0	0
11	12	13	15	14
	0	0	0	0
10	8	9	11	10
	0	0	0	0

Atención al orden: m3 va antes que m2; m7 antes que m6; etc. Mapa de 3 variables (A\BC) y de 4 variables (AB\CD).

3.2 Reglas para agrupar y leer

- Grupos de **2^k celdas** (1, 2, 4, 8, 16); un grupo de 2^k elimina k variables.
- Grupos **rectangulares** (sin L ni diagonales), **lo más grandes posible**, con bordes y esquinas enrollados.

- **Leer** un grupo: la variable constante en 1 → A; constante en 0 → A'; si cambia → desaparece. Un término por grupo.
- **Implicante**: cualquier grupo válido · **primo**: no se puede agrandar · **primo esencial**: único que cubre algún 1. Solución mínima = esenciales + los primos mínimos necesarios para cubrir el resto (puede no ser única).
- **POS**: se agrupan los 0 y la lectura se invierte (0 → A, 1 → A').

3.3 Ejemplo: F = Σm(0,1,2,5,8,9,10)



B'C' = m0, m1, m8, m9 (bordes superior e inferior)

B'D' = las 4 esquinas

A'C'D = m1, m5

SOP: F = B'C' + B'D' + A'C'D (3 términos, 7 literales)

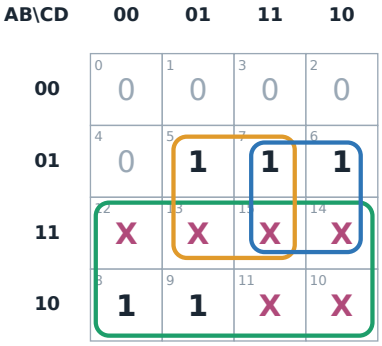
POS (agrupando 0): F = (A'+B')(B'+D)(C'+D') (6 literales).

→ Se construyen ambas y se elige por costo (literales, compuertas, CI).

Ejemplo propio (taller T1): F = Σm(1,3,4,6): m1+m3 → A'C; m4+m6 → AC'. Resultado: F = A'C + AC' = A ⊕ C (una XOR).

3.4 Condiciones indiferentes (X)

Combinaciones que nunca ocurren (p. ej. 1010–1111 en BCD). Notación: F = Σm(…) + d(…). **Como 1** si agrandan un grupo; **como 0** si no ayudan (ninguna X está obligada a cubrirse); **nunca solas** en un grupo (agregarían un término inútil).



BCD ≥ 5: F = Σm(5,6,7,8,9) + d(10…15)

Sin usar las X: AB'C' + A'BD + A'BC → **9 literales**, 3 CI (7404 + 7411 + 7432).

Con las X: **F = A + BD + BC** → **5 literales**, 2 CI (7408 + 7432).

A (filas 11 y 10), **BD**, **BC**.

3.5 Universalidad NAND y NOR

Función	Solo NAND	Solo NOR
NOT	(A·A)'	(A + A)'
AND	((AB)')'	(A' + B')'
OR	(A'·B')'	((A+B)')'

SOP mínima → NAND-NAND · POS mínima → NOR-NOR. Pasos (ejemplo F = A + BD + BC): (1) SOP mínima → (2) doble negación F = ((A + BD + BC)')' → (3) De Morgan: F = (A'·(BD)'·(BC)')' → (4) F = NAND(A', NAND(B,D), NAND(B,C)). El literal que entra solo se **complementa** (NAND como inversor). Mismos dos niveles de retardo, una sola familia (7400 + 7410 = 2 CI).

> **Ahorro:** el ahorro de chips vino de las **X**; la NAND no agrega chips y deja una sola familia en el inventario. El mínimo algebraico no siempre es el más barato.

Semana 4 · Circuitos aritméticos

4.1 Sumar un bit: semisumador y sumador completo

Dos tipos de columna: la columna 0 suma 2 bits (**semisumador**); las demás suman 3 bits, A, B y el acarreo que llega Cin (**sumador completo**).

Semisumador

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

S = A ⊕ B (XOR) · **C = A·B** (AND). Solo sirve para la columna 0: no recibe acarreo.

Sumador completo (contar los unos y escribirlo en binario)

A	B	Cin	# de 1	Cout	S
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	2	1	0
1	0	0	1	0	1
1	0	1	2	1	0
1	1	0	2	1	0
1	1	1	3	1	1

> **S = A ⊕ B ⊕ Cin** (1 si hay un número **impar** de unos; en el K-map es un «tablero de ajedrez», no hay grupos) · **Cout = AB + A·Cin + B·Cin** (1 si hay **al menos dos** unos; tres grupos de dos). No hace falta memorizar: tabla → K-map → expresión.

4.2 Sumador de 4 bits en cadena (ripple-carry) y retardo

- Cuatro sumadores completos; regla única: **Cout de uno → Cin del siguiente**. Ej.: 0110 + 0111 = 1101 (6 + 7 = 13), con acarreos que se encadenan.
- El acarreo avanza como una ola: cada etapa espera a la anterior. Con ≈10 ns por compuerta y ≈2 compuertas por etapa: **retardo ≈ 2·n·10 ns**.
- 4 bits → 80 ns · 16 bits → 320 ns · 32 bits → 640 ns. **Más bits → más lento.**

4.3 Restar con el mismo circuito

A − B = A + (−B), con −B = invertir B y sumar 1. Ej.: B = 0011 → inv 1100 → +1 = 1101 (−3); 0110 + 1101 = 1 0011 → 3 ✓ (se descarta el 1 sobrante).

XOR como inversor controlado: cada bit de B pasa por B ⊕ M. Con **M = 0** → B pasa igual → **SUMA**. Con **M = 1** → B se invierte → **RESTA**; además M se conecta a Cin del sumador 0 y aporta el +1. Un solo circuito suma o resta según M.

4.4 Overflow y comparador

- **Overflow** (4 bits con signo, −8 a +7): 0101 + 0100 = 1001 → −7 ✗. Se detecta con **V = C3 ⊕ C4** (acarreo que entra a la última etapa ≠ el que sale). **Cout = 1 NO significa error.** Ej.: 0101 + 0011 = 1000 = 8 → sin signo es correcto; con signo hay overflow (C3 = 1, C4 = 0).
- **Comparador de magnitud:** A = B → XNOR en cada bit y AND de todas. A > B → desde la izquierda, el primer bit en que difieren decide (2 bits: A gana en el bit alto, o empatan ahí y A gana en el bajo). Integrado 7485.
- **ALU** = sumador/restador + comparador + AND/OR/XOR + multiplexor (semana 5) → resultado de la instrucción.

4.5 Ejercicios resueltos del taller

Ejercicio	Resolución
1011 + 0110	Col.0: 1+0 = 1 (c 0) · Col.1: 1+1 = 0 (c 1) · Col.2: 0+1+1 = 0 (c 1) · Col.3: 1+0+1 = 0 (c 1) → **1 0001** = 17 (11 + 6).
0110 − 0011 (M = 1)	B invertido = 1100, Cin = 1 → 0110 + 1100 + 1 = 1 0011 → **0011 = 3**.
Retardo de 16 bits	2 · 16 · 10 ns = **320 ns**.
A = 10, B = 01	Bit alto: 1 > 0 → **A > B** es verdadero y **A = B** es falso.

Cierre · Resumen general y errores frecuentes

Fórmulas y reglas que más uso

Tema	Regla
Negativo (comp. a dos)	−B = invertir bits de B + 1 · rango n bits: -2^{n-1} ... $2^{n-1} - 1$
Canónicas	Σm = filas con 1 (OR de minitérminos) · ΠM = filas con 0 (AND de maxitérminos)
De Morgan	(AB)' = A' + B' · (A + B)' = A'B'
Adyacencia / absorción	XY + XY' = X · X + XY = X · X + X'Y = X + Y
Karnaugh	Grupos 2^k elimina k variables · SOP agrupa 1 · POS agrupa 0 (lectura invertida) · X como 1 solo si agranda
NAND / NOR	SOP → NAND-NAND · POS → NOR-NOR
Sumador	S = A ⊕ B ⊕ Cin · Cout = AB + A·Cin + B·Cin · retardo ≈ 2·n·10 ns
Resta / overflow	Resta: B ⊕ M y Cin = M · Overflow: V = C3 ⊕ C4

Errores que debo evitar

- Leer mal el enunciado: «al menos dos» ≠ «exactamente dos».
- Poner m2 antes que m3 en el mapa: el orden de columnas es 00, 01, **11, 10** (Gray).
- Agrupar en forma de L o en grupos que no son potencias de 2; olvidar los bordes y las esquinas.
- Dejar un grupo formado solo por X, o un primo que sobra (todos sus 1 ya están cubiertos).
- En POS, leer los grupos de 0 sin invertir (0 → A, 1 → A').
- Confundir Cout = 1 con overflow: en complemento a dos se usa C3 ⊕ C4.
- Dejar una entrada de un CI sin conectar: queda en la zona indefinida.

Conexión con lo que viene

Lo diseñado en la Unidad 1 es el **bloque combinacional** del proyecto integrador (semana 6, simulado en Logisim-evolution); en la Unidad 2 se agrega memoria (flip-flops) y en la Unidad 3 el sistema se describe en VHDL para la Basys 3.